

Lab 9 Solutions(Grp 1-8)

Vaccine and Recoveries :

```
def numVaccine(n):
```

```
    v1 = 0
```

```
    v2 = 1
```

```
    v3 = 1
```

```
    if n==1:
```

```
        return 0
```

```
    if n==2:
```

```
        return 1
```

```
    if n==3:
```

```
        return 1
```

```
    else:
```

```
        vacc_sum=0
```

```
        for x in range(4, n+1):
```

```
            vacc_sum = v1+v2+v3
```

```
            v1=v2
```

```
            v2=v3
```

```
            v3=vacc_sum
```

```
        return vacc_sum
```

```
def numRecovery(n):
```

```
    r1 = 0
```

```
    r2 = 1
```

```
    if n==1:
```

```
        return 0
```

```
    if n==2:
```

```
        return 1
```

```
    else:
```

```
        rec_sum=0
```

```
        for x in range(3, n+1):
```

```
            rec_sum = r1+r2
```

```
            r1=r2
```

```
            r2=rec_sum
```

```
        return rec_sum
```

```

try:
    n = int(input())
    assert n>0, "Not possible"
    diff = numVaccine(n)-numRecovery(n)
    print(diff)

except AssertionError as ae:
    print(ae)

```

Seconds past Midnight :

```

hr = int(input())
mn = int(input())

try:
    assert hr<=24, "Invalid"
    assert mn<=60, "Invalid"

    ts = (hr*60+mn)*60

    print(ts)

except AssertionError as ae:
    print(ae)

```

Reciprocity ;

```

raf, age = [int(x) for x in input().split()]

try:
    ras = 1/(raf-age)
    print(f'{ras:.2f}')

except:
    print("Ambivert")

```

Find factorial :

```
def isInteger(x):
    try :
        int(x);
        return True
    except ValueError:
        return False
def fact(n):
    ans = 1
    for i in range(2,n+1):
        ans *=i
    return ans;
n = int(input())
ans = ""
for i in range(n):
    x = input()
    if isInteger(x) and int(x)>=0:
        ans +=str(fact(int(x)))
    else :
        ans += "Invalid Operation"
    ans += "\n"
print(ans)
```

Tribonacci sequence :

```
n = int(input())
```

```
try:
    a1 = 0
    a2 = 1
    a3 = 1

    if (n==1):
        print(a1)
    elif(n==2):
        print(a2)
    elif (n==3):
        print(a3)
```

```

else:
    ## none of the base cases, then proceeding further
    for i in range(4, n+1):
        tribo_sum = a1+a2+a3
        a1=a2
        a2=a3
        a3=tribo_sum

    print(tribo_sum)

except:
    print("Not allowed")

```

Harmonic mean ;

```

lst = [int(i) for i in input().split()]
mean = 0
den = 0
try:
    for i in range(len(lst)):
        try:
            den += 1/lst[i]
        except ZeroDivisionError:
            pass
    mean = len(lst)/den
    print(round(mean,1))
except ZeroDivisionError:
    print(0.0)

```

Index bound:

```

ls = [int(x) for x in input().split()]

n = int(input())

try:
    assert n>=0, -1
    assert n<len(ls), -1

```

```
print(ls[n])
```

```
except AssertionError as ae:  
    print(ae)
```